

Министерство образования и науки Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет _____ ИТР
Кафедра _____ ИС

КУРСОВАЯ РАБОТА

по курсу Прикладная разработка на Java
на тему: разработка веб-приложения «Аукцион»

Руководитель

_____ к. т. н. каф. ИС
(уч. степень, звание)

_____ Метелкин А.С.
(фамилия, инициалы)

_____ (подпись) _____ (дата)

Члены комиссии

Студент _____ ИС-123
(группа)

_____ (подпись) _____ (Ф.И.О.)

_____ Никоноров Д.А.
(фамилия, инициалы)

_____ (подпись) _____ (Ф.И.О.)

_____ (подпись) _____ (дата)

В курсовой работе отражено создание веб-приложения «Аукцион». Основной целью проекта является разработка программного обеспечения, которое реализует возможность проведения онлайн-аукционов, создания лотов, размещения ставок и управления пользователями с различными ролями.

Программа предоставляет удобный веб-интерфейс, обеспечивающий взаимодействие пользователя с системой через страницы регистрации, авторизации, просмотра аукционов, создания лотов, размещения ставок, личного кабинета, панели владельца и административной панели. В приложении реализована работа с базой данных, что позволяет хранить информацию о пользователях, аукционах, ставках и статусах торгов.

Проект разработан на языке программирования Java с использованием фреймворка Spring Boot. Для организации доступа к данным используется Spring Data JPA, для обеспечения безопасности — Spring Security. Пользовательский интерфейс реализован с использованием нескольких шаблонизаторов, что соответствует требованию реализации трёх разных движков web-интерфейса.

СОДЕРЖАНИЕ

1 АНАЛИЗ ТЕХНИЧЕСКОГО ЗАДАНИЯ	7
1.1 ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ.....	8
1.2 НЕФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ.....	10
1.3 ВЫБОР ТЕХНОЛОГИИ РАЗРАБОТКИ	11
2 ПРОЕКТИРОВАНИЕ ПРОГРАММЫ	14
2.1 ОБЩАЯ АРХИТЕКТУРА ПРОГРАММЫ	14
2.2 ОПИСАНИЕ МОДУЛЕЙ СИСТЕМЫ.....	15
2.3 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ	18
2.4 ПРОЕКТИРОВАНИЕ РОЛЕЙ ПОЛЬЗОВАТЕЛЕЙ	19
2.5 ПРОЕКТИРОВАНИЕ СТАТУСОВ АУКЦИОНА.....	20
2.6 ИСПОЛЬЗОВАНИЕ ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ	20
3 РАЗРАБОТКА ПРИЛОЖЕНИЯ.....	22
3.1 РЕАЛИЗАЦИЯ МОДЕЛИ ДАННЫХ	22
3.2 РЕАЛИЗАЦИЯ ДОСТУПА К ДАННЫМ.....	23
3.3 РЕАЛИЗАЦИЯ РЕГИСТРАЦИИ И АВТОРИЗАЦИИ	25
3.4 РЕАЛИЗАЦИЯ СИСТЕМЫ РОЛЕЙ	27
3.5 РЕАЛИЗАЦИЯ АУКЦИОНОВ	28
3.6 РЕАЛИЗАЦИЯ СИСТЕМЫ СТАВОК.....	29
3.7 РЕАЛИЗАЦИЯ ЛИЧНОГО КАБИНЕТА	30
3.8 ПАНЕЛЬ ВЛАДЕЛЬЦА	31
3.9 АДМИНИСТРАТИВНАЯ ПАНЕЛЬ	32
3.10 РЕАЛИЗАЦИЯ ТРЁХ WEB-ИНТЕРФЕЙСОВ	33
4 ТЕСТИРОВАНИЕ	35
4.1 ТЕСТИРОВАНИЕ МОДУЛЯ ПОЛЬЗОВАТЕЛЕЙ И РЕГИСТРАЦИИ.....	35
4.2 ТЕСТИРОВАНИЕ МОДУЛЯ АУКЦИОНОВ	37
4.3 ТЕСТИРОВАНИЕ МОДУЛЯ СТАВОК.....	39
4.4 ТЕСТИРОВАНИЕ ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ И ИНТЕРФЕЙСОВ	41
4.5 ТЕСТИРОВАНИЕ ФАБРИКИ КАРТОЧЕК АУКЦИОНОВ.....	43
4.6 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ.....	44
ЗАКЛЮЧЕНИЕ	47
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	48

					МИВУ 09.03.02 - 00.000 ПЗ							
Изм.	Лист	№ докум.	Подп.	Дата	разработка веб-приложения “Аукцион”			Лит.	Лист	Листов		
Разраб.	Никоноров Д.А.							у		4	48	
Пров.	Метелкин А.С.							МИ ВлГУ ИС-123				
Н. контр.												
Утв.												

Введение

В современном мире веб-приложения широко применяются для автоматизации различных процессов, связанных с обработкой информации, взаимодействием пользователей, хранением данных и предоставлением удалённого доступа к функциональности системы. Одним из примеров таких приложений являются онлайн-аукционы, которые позволяют пользователям размещать лоты, просматривать доступные предложения, участвовать в торгах и делать ставки через web-интерфейс.

Данный курсовой проект посвящён разработке веб-приложения «Аукцион» на языке программирования Java. Выбранная тема является актуальной, так как она объединяет в себе несколько важных направлений прикладной разработки: создание серверной части приложения, работу с базой данных, реализацию пользовательского интерфейса, настройку авторизации, разграничение ролей и обработку бизнес-логики.

Целью курсовой работы является создание полноценного веб-приложения для проведения аукционов, в котором пользователи могут регистрироваться, авторизовываться, просматривать активные аукционы и размещать ставки. Также в системе должны быть предусмотрены роли владельца и администратора. Владелец получает возможность создавать аукционы и управлять ими, а администратор — контролировать работу системы и управлять данными.

В соответствии с техническим заданием при разработке необходимо использовать паттерны проектирования, интерфейсы для организации структуры программы и взаимодействия модулей, базу данных для хранения информации, систему ролей, а также три разных движка web-интерфейса. Эти требования определяют архитектуру проекта и позволяют продемонстрировать применение современных подходов к разработке Java-приложений.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подп.	Дата		

В рамках курсовой работы рассматриваются основные этапы создания программного продукта: анализ технического задания, выбор технологий, проектирование архитектуры, реализация модулей приложения и тестирование разработанных компонентов. Особое внимание уделяется разделению приложения на слои, что позволяет сделать проект более понятным, расширяемым и удобным для сопровождения.

Разработанное приложение построено на основе фреймворка Spring Boot. Для работы с базой данных используется Spring Data JPA, для обеспечения безопасности и разграничения доступа — Spring Security. Пользовательский интерфейс реализуется с использованием шаблонизаторов, что позволяет формировать web-страницы и отображать данные пользователю в удобном виде.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подп.	Дата		

1 Анализ технического задания

Разработка веб-приложения «Аукцион» представляет собой задачу создания серверного приложения, предназначенного для проведения онлайн-торгов. Пользователь должен иметь возможность зарегистрироваться, войти в систему, просматривать список доступных аукционов и участвовать в торгах путём размещения ставок. Для владельцев аукционов должна быть предусмотрена возможность создания собственных лотов и управления ими. Для администратора должна быть реализована панель управления пользователями и аукционами.

Согласно техническому заданию, приложение должно быть разработано с использованием языка программирования Java и базы данных для хранения информации. В проекте необходимо реализовать систему ролей, использовать интерфейсы для организации взаимодействия модулей программы, применить паттерны проектирования, а также разработать три разных варианта web-интерфейса.

Одной из основных особенностей проекта является реализация принципа «три по три». В приложении используются три web-движка, три основных паттерна проектирования и три роли пользователей. Дополнительно в программной логике предусмотрены три статуса аукциона, которые отражают состояние торгов.

В качестве трёх web-движков используются Thymeleaf, Mustache и FreeMarker. Каждый из них применяется для формирования HTML-страниц приложения. Все три варианта интерфейса работают с одними и теми же данными и используют общую программную логику.

В проекте применяются три паттерна проектирования: Facade, Factory и Strategy. Паттерн Facade используется для объединения операций сервисов и упрощения работы контроллеров. Паттерн Factory применяется для создания карточек аукционов. Паттерн Strategy используется для проверки правил размещения ставок.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подп.	Дата		

В системе предусмотрены три роли пользователей: USER, OWNER и ADMIN. Роль USER назначается обычному пользователю и позволяет участвовать в торгах. Роль OWNER предназначена для владельца, который может создавать и закрывать собственные аукционы. Роль ADMIN используется для администратора, который может управлять пользователями и закрывать любые аукционы.

Для аукционов предусмотрены три статуса: DRAFT, ACTIVE и CLOSED. Статус DRAFT означает, что аукцион создан, но время его начала ещё не наступило. Статус ACTIVE означает, что аукцион активен и пользователи могут делать ставки. Статус CLOSED означает, что аукцион завершён и новые ставки больше не принимаются.

Таким образом, разрабатываемое приложение должно включать регистрацию и авторизацию пользователей, работу с ролями, хранение данных в базе, создание аукционов, размещение ставок, управление статусами и отображение страниц через три разных шаблонизатора.

1.1 Функциональные требования

Веб-приложение «Аукцион» должно обеспечивать регистрацию новых пользователей. При регистрации пользователь вводит логин, электронную почту и пароль. После успешной регистрации данные сохраняются в базе данных, пароль шифруется, а пользователю назначается роль USER.

Приложение должно поддерживать авторизацию пользователей. При входе в систему пользователь вводит логин и пароль. После успешной проверки данных система предоставляет доступ к функциям приложения в зависимости от роли пользователя.

В системе должны быть реализованы три роли: USER, OWNER и ADMIN. Пользователь с ролью USER должен иметь возможность просматривать аукционы, делать ставки, открывать личный кабинет, просматривать историю своих ставок и список выигранных аукционов.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						8
Изм.	Лист	№ докум.	Подп.	Дата		

Пользователь с ролью OWNER должен иметь возможность создавать новые аукционы, просматривать созданные им лоты, отслеживать ставки по своим аукционам и закрывать собственные аукционы. При этом владелец не должен иметь возможность делать ставки на свои собственные лоты.

Пользователь с ролью ADMIN должен иметь возможность открывать административную панель, просматривать список пользователей, изменять роли, блокировать и разблокировать учётные записи, а также закрывать любые аукционы.

Приложение должно обеспечивать создание аукционов. При создании аукциона владелец указывает название, описание, стартовую цену, дату начала и дату окончания торгов. После создания аукцион сохраняется в базе данных и получает статус в зависимости от времени начала.

Система должна предоставлять возможность просмотра списка аукционов. На странице списка должны отображаться основные сведения о лоте: название, описание, стартовая цена, текущая цена, статус, дата начала и дата окончания торгов.

Одной из основных функций приложения является размещение ставок. Пользователь может сделать ставку только на активный аукцион. Сумма новой ставки должна быть больше текущей цены. Если ставка не соответствует требованиям, она не должна сохраняться.

Приложение должно поддерживать закрытие аукционов. После закрытия аукцион получает статус CLOSED, а размещение новых ставок становится невозможным. Победителем считается пользователь, сделавший максимальную ставку.

Также приложение должно иметь три варианта web-интерфейса с использованием Thymeleaf, Mustache и FreeMarker.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						9
Изм.	Лист	№ докум.	Подп.	Дата		

1.2 Нефункциональные требования

Приложение должно быть разработано на языке Java с использованием объектно-ориентированного подхода. Основные сущности системы должны быть представлены в виде классов, а взаимодействие между частями программы должно быть организовано через отдельные модули.

Код проекта должен быть разделён на уровни. В приложении должны быть выделены контроллеры, фасад, сервисы, репозитории, модели данных и шаблоны пользовательского интерфейса. Такое разделение упрощает сопровождение проекта и позволяет изменять отдельные части программы без изменения всей структуры.

При разработке должны использоваться интерфейсы. Интерфейсы позволяют задать общий набор операций и отделить описание действий от конкретной реализации. В проекте интерфейсы применяются для сервисов, фасада, стратегии проверки ставок и репозиториях.

Данные приложения должны храниться в базе данных. Информация о пользователях, аукционах и ставках не должна теряться после завершения работы программы. Для хранения данных используется RED Database / Firebird.

Приложение должно обеспечивать безопасность данных пользователей. Пароли должны храниться в зашифрованном виде. Доступ к защищённым страницам должен предоставляться только авторизованным пользователям с соответствующей ролью.

Пользовательский интерфейс должен быть понятным и удобным. Пользователь должен иметь возможность быстро перейти к регистрации, входу, просмотру аукционов, созданию лота, размещению ставки и личному кабинету.

Приложение должно корректно обрабатывать ошибочные ситуации. При вводе неверных данных, попытке сделать некорректную ставку или обращении к недоступной странице система должна выводить сообщение об ошибке и не завершать работу аварийно.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						10
Изм.	Лист	№ докум.	Подп.	Дата		

1.3 Выбор технологии разработки

Для реализации веб-приложения необходимо выбрать технологию, позволяющую создавать серверную часть приложения, работать с базой данных, реализовывать авторизацию пользователей, обрабатывать HTTP-запросы и формировать web-интерфейс. В качестве возможных вариантов были рассмотрены Spring Boot, Jakarta EE и JavaFX.

1.3.1 Spring Boot

Spring Boot является современным фреймворком для разработки Java-приложений. Он предоставляет удобные средства для создания web-приложений, обработки запросов, настройки маршрутов, взаимодействия с базой данных и реализации безопасности.

Одним из преимуществ Spring Boot является автоматическая конфигурация. Благодаря этому можно быстрее создать рабочее приложение без необходимости вручную настраивать большое количество компонентов.

Spring Boot хорошо интегрируется с другими модулями экосистемы Spring. Для работы с базой данных используется Spring Data JPA. Для настройки авторизации и разграничения доступа применяется Spring Security. Для отображения страниц могут использоваться различные шаблонизаторы.

Spring Boot подходит для данного проекта, так как веб-приложение «Аукцион» требует работы с пользователями, ролями, базой данных, web-страницами, безопасностью и программной логикой.

1.3.2 Jakarta EE

Jakarta EE является платформой для разработки корпоративных Java-приложений. Она предоставляет набор спецификаций для создания серверных

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						11
Изм.	Лист	№ докум.	Подп.	Дата		

систем, включая сервлеты, работу с базами данных, обработку запросов и управление компонентами.

Данная технология обладает широкими возможностями и может использоваться для создания сложных информационных систем. Однако для учебного проекта Jakarta EE может оказаться более сложной в настройке и разработке. Создание приложения на этой платформе требует большего количества ручной конфигурации.

По сравнению со Spring Boot, Jakarta EE менее удобна для быстрого создания приложения с авторизацией, базой данных и web-интерфейсом. Поэтому данная технология рассматривалась как возможный, но менее предпочтительный вариант.

1.3.3 JavaFX

JavaFX предназначен для разработки настольных приложений с графическим интерфейсом. Он позволяет создавать окна, кнопки, таблицы, формы и другие элементы интерфейса.

Однако разрабатываемый проект должен быть web-приложением, доступным через браузер. Для такой задачи JavaFX не является подходящим вариантом, так как он не предназначен для создания серверных web-систем.

Использование JavaFX потребовало бы разработки отдельного настольного приложения, что не соответствует техническому заданию. Поэтому JavaFX не был выбран для реализации проекта «Аукцион».

1.3.4 Итог и выбор

Для разработки веб-приложения «Аукцион» был выбран Spring Boot. Он предоставляет наиболее подходящий набор инструментов для выполнения требований технического задания: создание web-приложения, работа с базой

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						12
Изм.	Лист	№ докум.	Подп.	Дата		

данных, реализация авторизации, разграничение ролей, использование шаблонизаторов и построение модульной структуры.

В отличие от Jakarta EE, Spring Boot позволяет быстрее настроить приложение и сосредоточиться на реализации основной логики работы системы. В отличие от JavaFX, он ориентирован именно на web-разработку и взаимодействие через браузер.

Таким образом, Spring Boot является оптимальным выбором для реализации данного проекта. В сочетании со Spring Data JPA, Spring Security, Hibernate, Maven и серверными шаблонизаторами он позволяет создать полноценное веб-приложение для проведения онлайн-аукционов.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						13
Изм.	Лист	№ докум.	Подп.	Дата		

2 Проектирование программы

2.1 Общая архитектура программы

Перед началом разработки была определена общая структура веб-приложения. Для удобства сопровождения и расширения проект разделён на несколько уровней. Каждый уровень выполняет свою задачу и взаимодействует с другими уровнями через классы и интерфейсы.

Общая архитектура приложения имеет следующий вид:

Пользователь → Контроллеры → Фасад → Сервисы → Репозитории → База данных^[4]

Уровень представления отвечает за отображение HTML-страниц пользователю. В проекте он реализован с использованием трёх шаблонизаторов: Thymeleaf, Mustache и FreeMarker.

Уровень контроллеров отвечает за приём запросов от пользователя. Контроллеры получают данные из формы или REST-запроса, передают их дальше для обработки и возвращают результат пользователю.

Фасад используется как промежуточный слой между контроллерами и сервисами. Он объединяет вызовы нескольких сервисов и формирует DTO-объекты, удобные для отображения на страницах.

Сервисный уровень содержит основную программную логику приложения. В нём выполняется регистрация пользователей, создание аукционов, размещение ставок, изменение статусов и управление пользователями.

Уровень репозитория отвечает за доступ к базе данных. Репозитории используют Spring Data JPA и предоставляют методы для поиска, сохранения и изменения данных.

База данных хранит информацию о пользователях, аукционах и ставках. Общая архитектура веб-приложения показана на рисунке 1.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						14
Изм.	Лист	№ докум.	Подп.	Дата		

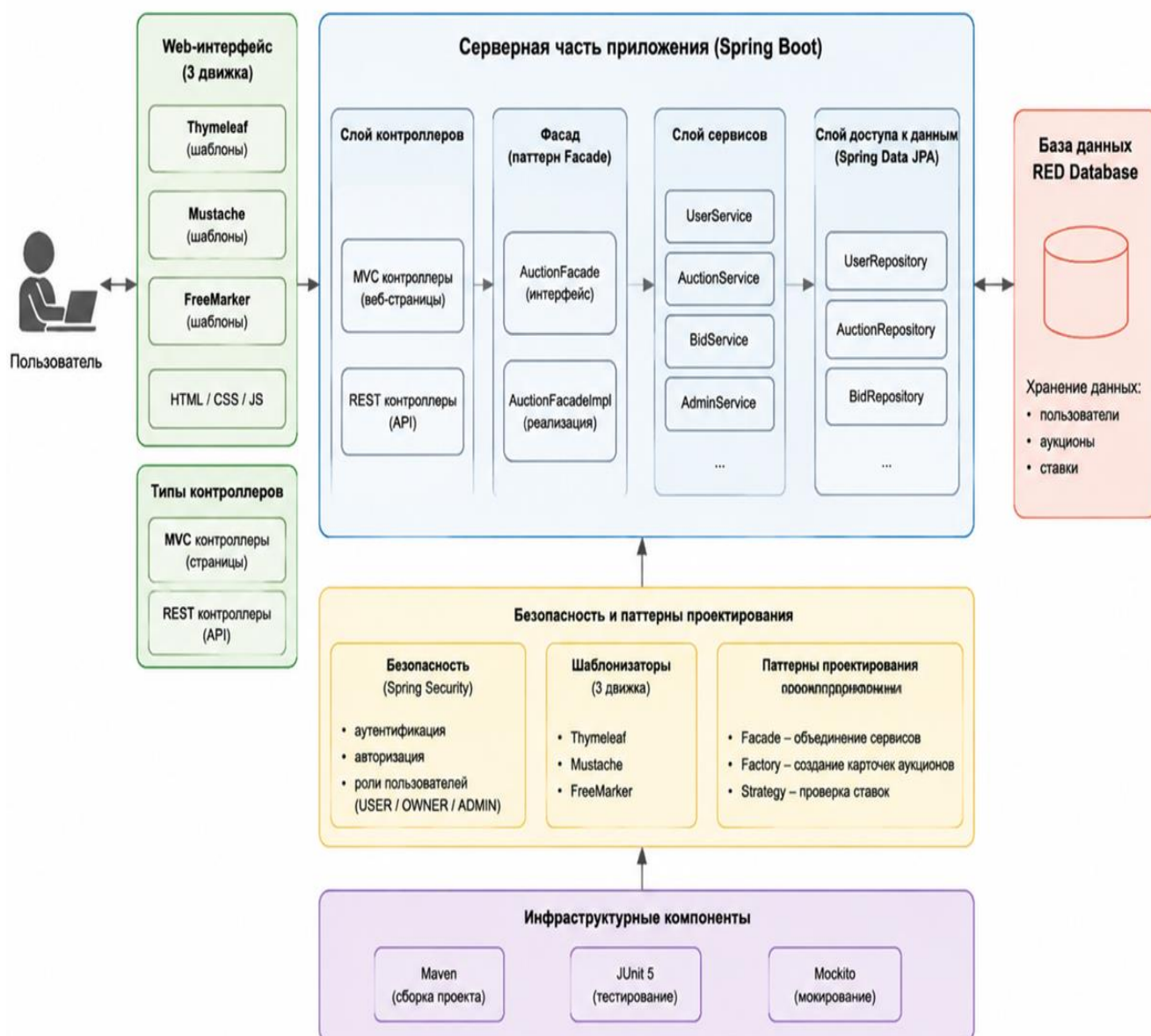


Рисунок 1 – Общая архитектура веб-приложения «Аукцион»

2.2 Описание модулей системы

Проект веб-приложения «Аукцион» имеет модульную структуру. Исходный код разделён на функциональные модули, каждый из которых выполняет определённую задачу и взаимодействует с другими модулями через интерфейсы и DTO.

Конфигурационный модуль содержит классы для настройки приложения. В нём находятся компоненты для конфигурации безопасности (Spring Security), инициализации демонстрационных пользователей, настройки базы данных RED Database, а также определения параметров web-шаблонизаторов.

Модуль контроллеров отвечает за обработку запросов пользователей. Контроллеры делятся на MVC-контроллеры для web-страниц и REST-контроллеры для API. Они принимают запросы, передают данные в фасад или сервисы и возвращают результаты пользователю.

Модуль фасада реализован через интерфейс AuctionFacade и его реализацию AuctionFacadeImpl. Фасад объединяет работу сервисов и предоставляет контроллерам удобные методы для получения данных и обработки действий пользователей. Здесь же реализованы паттерны проектирования Facade, Factory и Strategy, а также создаются DTO для передачи информации между слоями.

Сервисный модуль содержит классы UserService, AuctionService, BidService. Сервисы реализуют основную логику приложения: регистрацию и авторизацию пользователей, создание и управление аукционами, размещение ставок, обработку правил ставок. Сервисы используют репозитории для взаимодействия с базой данных и фасад для упрощения вызовов в контроллерах.

Модуль репозиториев включает интерфейсы UserRepository, AuctionRepository, BidRepository. Репозитории предоставляют методы для доступа к таблицам базы данных, поиска, сохранения и обновления информации. Все операции с данными выполняются через сервисы и фасад, что обеспечивает модульность и отделение логики приложения от хранения данных.

Модуль моделей и DTO содержит основные сущности (User, Auction, Bid) и объекты передачи данных (AuctionCardDto, BidDto и др.). Сущности отражают таблицы базы данных, а DTO используются для передачи информации между слоями приложения и web-интерфейсом.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						16
Изм.	Лист	№ докум.	Подп.	Дата		

Модуль шаблонизаторов содержит три варианта web-интерфейса: Thymeleaf, Mustache и FreeMarker. Они взаимодействуют с сервисами через фасад и обеспечивают отображение страниц для пользователей, владельцев и администраторов.

Модуль безопасности включает классы для аутентификации и авторизации пользователей, работу с ролями и разграничение доступа к страницам и API. В проекте это реализовано через Spring Security с использованием пользовательского сервиса для загрузки данных пользователя.

Модуль утилит содержит вспомогательные классы для обработки ошибок, форматирования данных и реализации общих функций, используемых в разных частях приложения. Структура проекта и расположение модулей показаны на рисунке 2.

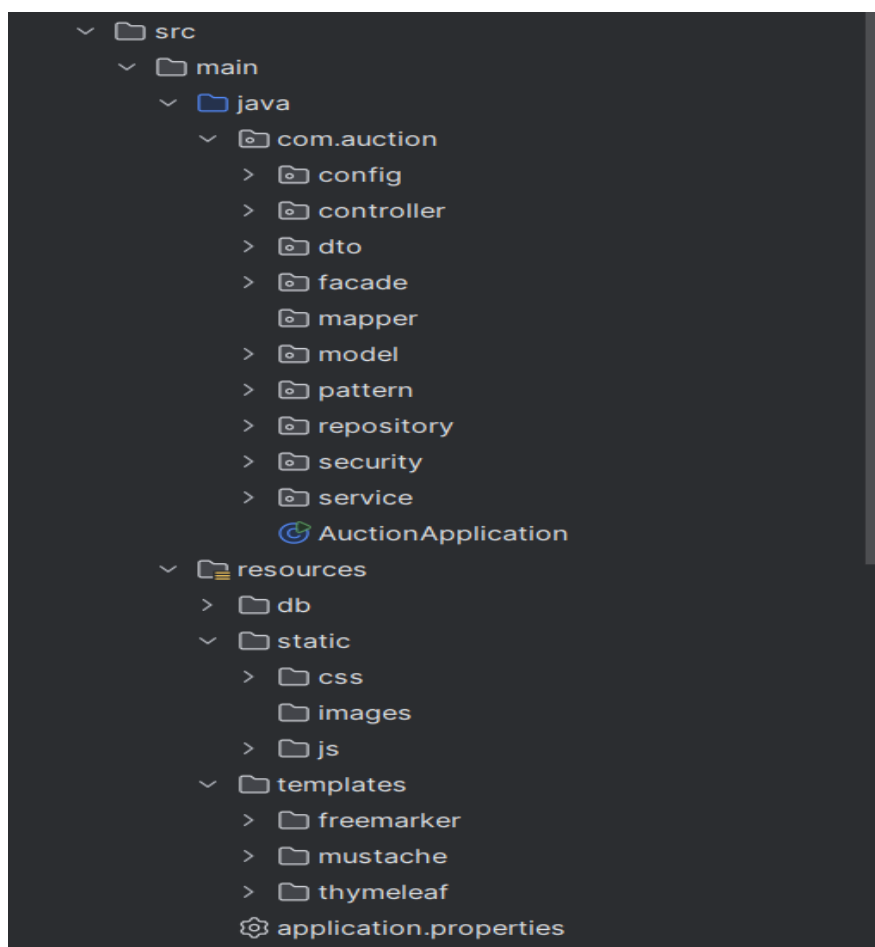


Рисунок 2 – Структура проекта в среде разработки

2.3 Проектирование базы данных

Для хранения данных используется реляционная база данных RED Database / Firebird^[3]. В базе данных предусмотрены три основные таблицы:

- APP_USER;
- AUCTION;
- BID.

Таблица APP_USER предназначена для хранения информации о пользователях. В ней содержатся идентификатор пользователя, логин, электронная почта, пароль, роль и признак активности учётной записи.

Таблица AUCTION предназначена для хранения информации об аукционах. В ней содержатся идентификатор аукциона, название, описание, стартовая цена, дата начала, дата окончания, статус и идентификатор владельца.

Таблица BID предназначена для хранения информации о ставках. В ней содержатся идентификатор ставки, сумма ставки, время размещения, идентификатор пользователя и идентификатор аукциона.

Связи между таблицами имеют следующий вид:

- один пользователь может создать несколько аукционов;
- один пользователь может сделать несколько ставок;
- один аукцион может иметь несколько ставок;
- ставка относится к одному пользователю и одному аукциону.

В базе данных используются ограничения целостности. Для таблицы пользователей задана уникальность логина и электронной почты. Для роли пользователя установлено ограничение допустимых значений: USER, OWNER, ADMIN. Для статуса аукциона установлено ограничение допустимых значений: DRAFT, ACTIVE, CLOSED. Для ставки установлено ограничение, согласно которому сумма должна быть больше нуля. ER-диаграмма базы данных представлена на рисунке 3.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						18
Изм.	Лист	№ докум.	Подп.	Дата		

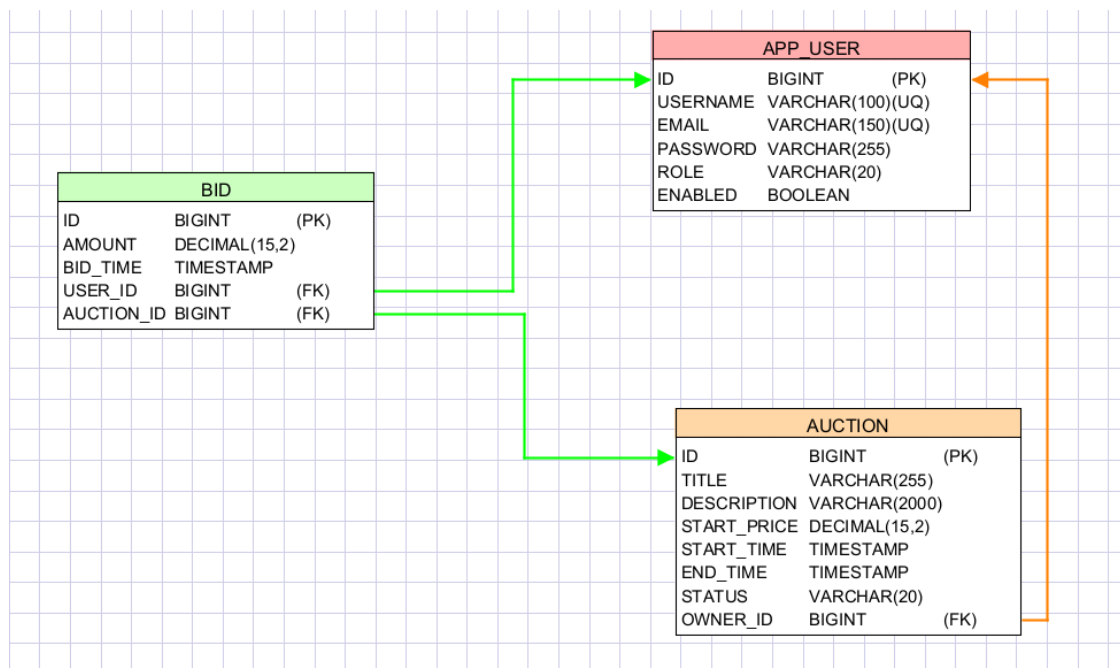


Рисунок 3 – ER-диаграмма базы данных приложения

2.4 Проектирование ролей пользователей

В приложении предусмотрены три роли пользователей. Роли используются для разграничения доступа к функциям системы.

Роль USER назначается пользователю автоматически при регистрации. Такой пользователь может участвовать в торгах, но не может создавать собственные аукционы и открывать административные страницы.

Роль OWNER предназначена для владельца лотов. Такой пользователь может создавать аукционы и закрывать только те аукционы, которые принадлежат ему.

Роль ADMIN предназначена для администратора системы. Администратор имеет доступ к панели управления пользователями и аукционами. Для защиты системы предусмотрено ограничение: администратор не может изменить самого себя и не может изменить другого администратора.

2.5 Проектирование статусов аукциона

Для описания состояния аукциона используется перечисление AuctionStatus. В приложении предусмотрены три статуса: DRAFT, ACTIVE и CLOSED.

Статус DRAFT означает, что аукцион создан, но время его начала ещё не наступило. Такой аукцион хранится в системе, но ставки по нему не принимаются.

Статус ACTIVE означает, что аукцион активен и пользователи могут делать ставки. При размещении ставки система проверяет, что аукцион находится именно в этом состоянии.

Статус CLOSED означает, что аукцион завершён. После закрытия аукциона новые ставки не принимаются. Победителем считается пользователь, который сделал максимальную ставку.

Статусы позволяют организовать жизненный цикл аукциона и избежать некорректных действий, например размещения ставок на завершённый лот.

2.6 Использование паттернов проектирования

В проекте используются три основных паттерна проектирования: Facade, Factory и Strategy^[1].

Паттерн Facade реализован с помощью интерфейса AuctionFacade и класса AuctionFacadeImpl. Фасад предоставляет контроллерам единый набор методов для работы с аукционами, ставками, пользователями и административными функциями. Это позволяет уменьшить количество зависимостей в контроллерах и сделать их проще.

Паттерн Factory реализован в классе AuctionCardFactory. Этот класс отвечает за создание объекта AuctionCardDto на основе сущности Auction и текущей цены. Использование фабрики позволяет вынести создание DTO в отдельный компонент и не дублировать этот код в разных частях приложения.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						20
Изм.	Лист	№ докум.	Подп.	Дата		

Паттерн Strategy реализован через интерфейс BidValidationStrategy и класс DefaultBidValidationStrategy. Данная стратегия проверяет корректность ставки. Она проверяет, что аукцион активен, время торгов не истекло, пользователь не является владельцем лота, а новая ставка больше текущей цены.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						21
Изм.	Лист	№ докум.	Подп.	Дата		

3 Разработка приложения

3.1 Реализация модели данных

Модель данных приложения включает три основные сущности: User, Auction и Bid.

Сущность User описывает пользователя системы. Она связана с таблицей APP_USER. В классе хранятся логин, электронная почта, пароль, роль и признак активности. Роль пользователя задаётся перечислением Role, которое содержит значения USER, OWNER и ADMIN.

Сущность Auction описывает аукцион. Она связана с таблицей AUCTION. В классе хранятся название, описание, стартовая цена, дата начала, дата окончания, статус и владелец. Статус задаётся перечислением AuctionStatus, которое содержит значения DRAFT, ACTIVE и CLOSED.

Сущность Bid описывает ставку пользователя. Она связана с таблицей BID. В классе хранятся сумма ставки, время размещения, пользователь и аукцион.

Между сущностями настроены связи. Один пользователь может быть владельцем нескольких аукционов. Один пользователь может сделать несколько ставок. Один аукцион может иметь несколько ставок. Такие связи позволяют получать историю торгов, определять текущую цену и находить победителя закрытого аукциона. Основные сущности приложения User, Auction и Bid показаны на рисунке 4.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						22
Изм.	Лист	№ докум.	Подп.	Дата		

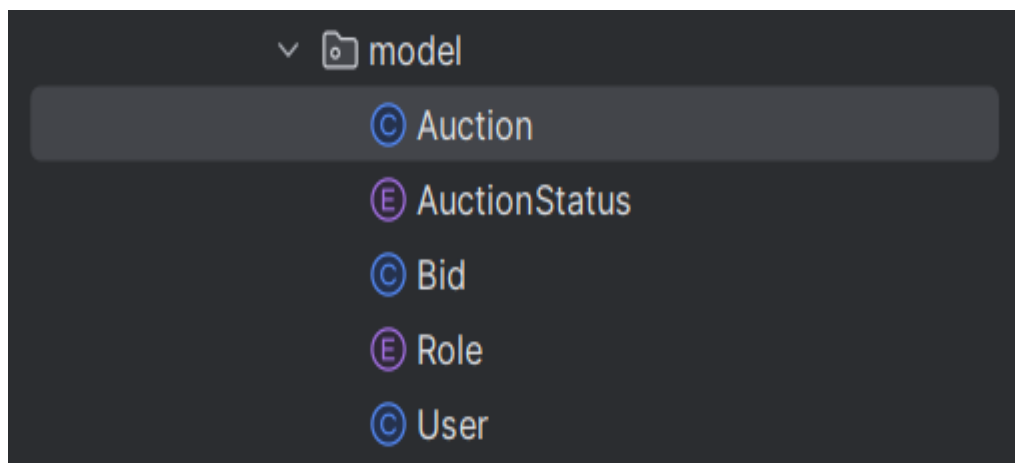


Рисунок 4 – Основные сущности приложения

3.2 Реализация доступа к данным

Доступ к данным реализован через репозитории Spring Data JPA^[2]. В проекте созданы следующие репозитории: UserRepository, AuctionRepository и BidRepository.

UserRepository используется для работы с пользователями. В нём предусмотрены методы поиска пользователя по логину, проверки существования логина и проверки существования электронной почты.

AuctionRepository используется для работы с аукционами. В нём предусмотрены методы получения аукционов, поиска по статусу и получения аукционов конкретного владельца.

BidRepository используется для работы со ставками. В нём предусмотрены методы получения максимальной ставки по аукциону, получения всех ставок по аукциону и получения ставок конкретного пользователя.

Использование репозиторий позволяет отделить работу с базой данных от остальной части приложения. Сервисы не выполняют SQL-запросы напрямую, а обращаются к репозиториям через методы Java-интерфейсов. Схема взаимодействия реализаций программы представлена на рисунке 5. Схема взаимодействия интерфейсов программы представлена на рисунке 6

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						23
Изм.	Лист	№ докум.	Подп.	Дата		

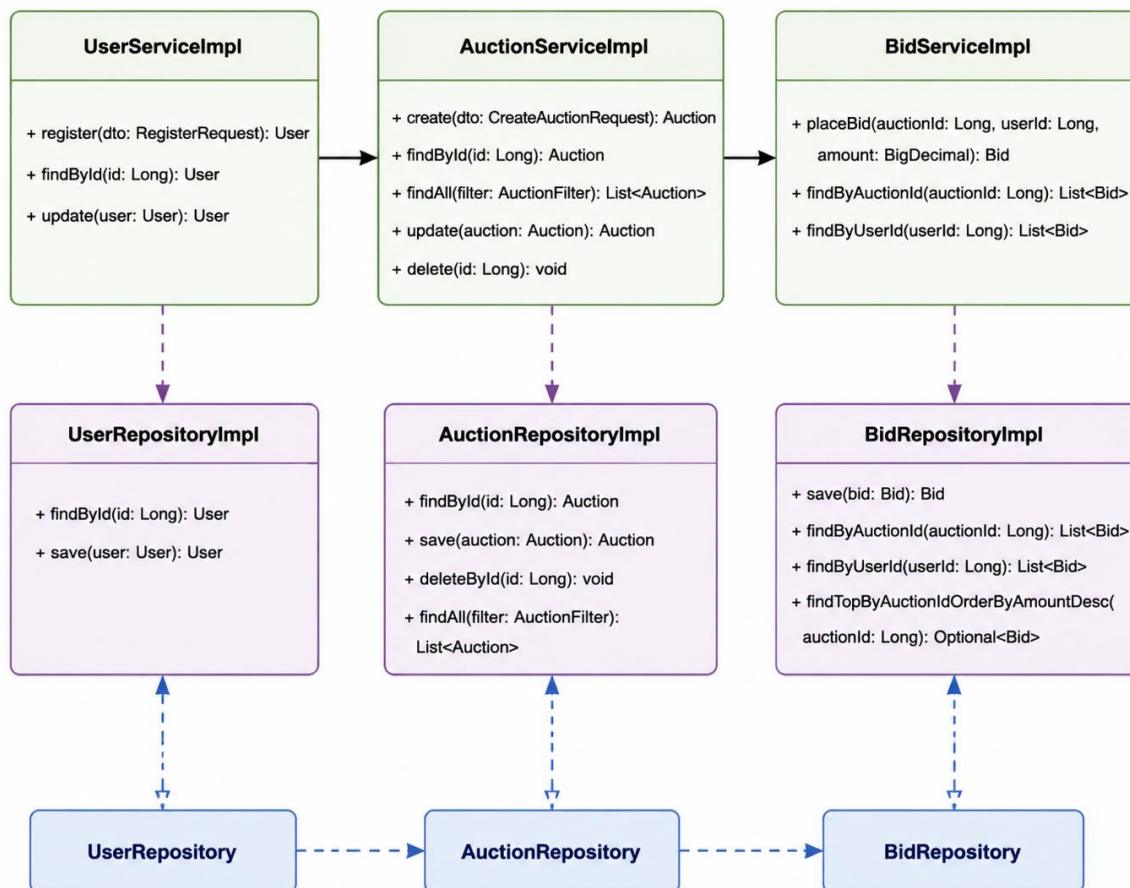


Рисунок 5 – Схема взаимодействия реализаций

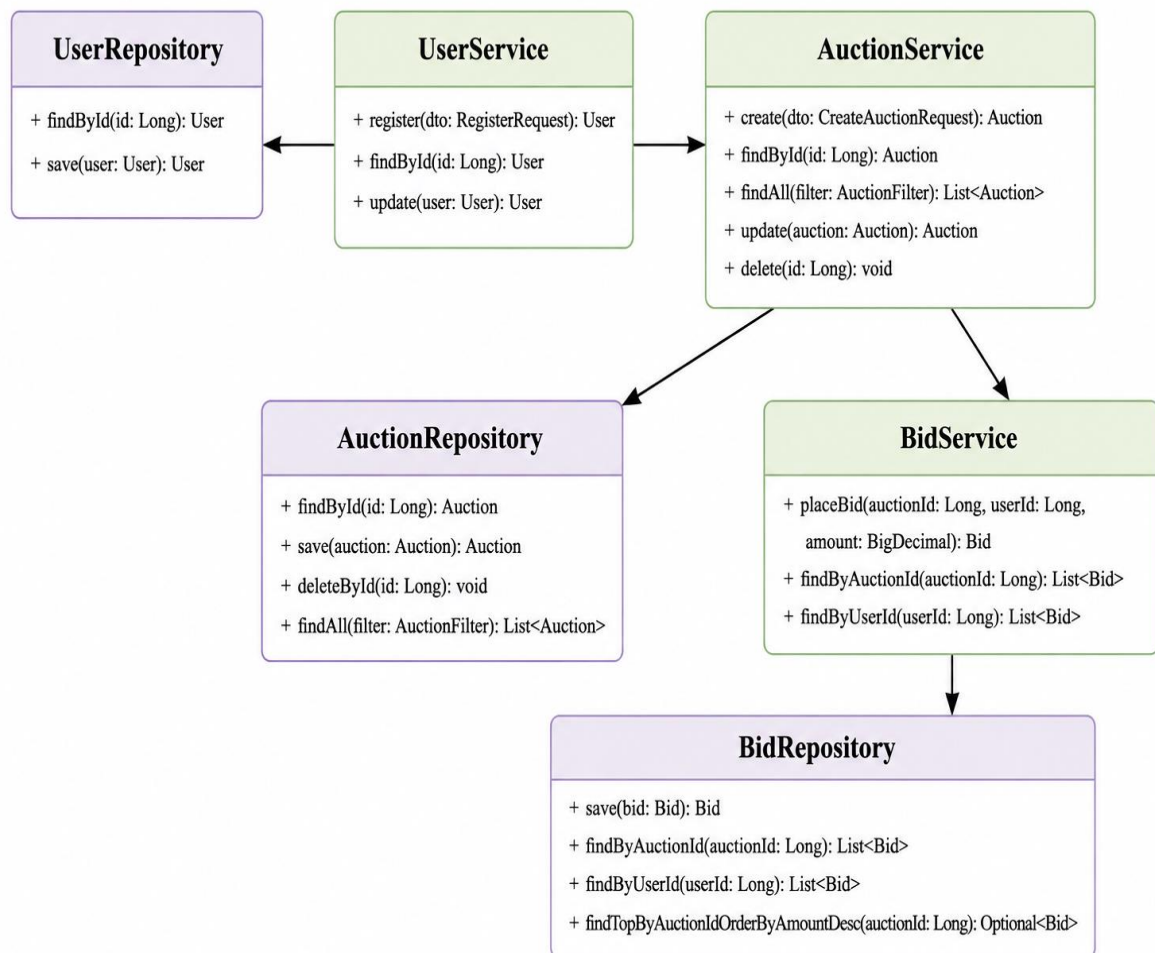


Рисунок 6 - Схема взаимодействия интерфейсов

3.3 Реализация регистрации и авторизации

Регистрация пользователя реализована в сервисе `UserServiceImpl`. При регистрации система получает объект `RegisterRequest`, содержащий логин, электронную почту и пароль.

Перед созданием нового пользователя выполняются проверки. Система проверяет, что указанный логин ещё не используется, а электронная почта не принадлежит другому пользователю. Если такие данные уже существуют, регистрация отклоняется.

После успешной проверки создаётся объект User. Пользователю назначается роль USER, учётная запись устанавливается в активное состояние, а пароль шифруется с помощью PasswordEncoder.

Авторизация реализована через Spring Security^[1]. Для загрузки пользователя используется класс AuctionUserDetailsService. Он находит пользователя в базе данных по логину и передаёт Spring Security данные о пароле, активности учётной записи и роли.

После успешного входа пользователь получает доступ к страницам приложения. Если вход выполнен с ошибкой, пользователь возвращается на страницу авторизации с сообщением о неверном логине или пароле. Интерфейс регистрации и авторизации представлен на рисунках 7–8.

Рисунок 7 – Страница регистрации

Рисунок 8 – Страница авторизации

3.4 Реализация системы ролей

Система ролей в приложении основывается на трех типах пользователей: обычный пользователь, владелец аукционов и администратор. Каждый тип пользователя имеет свои права и ограничения на выполнение действий в системе.

Роль пользователя предоставляет доступ только к просмотру активных аукционов и размещению ставок. Владелец может создавать аукционы, просматривать свои лоты, управлять статусами аукционов и закрывать торги. Администратор имеет полный доступ к системе, включая управление пользователями, тестами и контролем всех операций.

Роль каждого пользователя задается при регистрации и сохраняется в базе данных. При авторизации система использует Spring Security для проверки роли пользователя и ограничения доступа к соответствующим страницам.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						27
Изм.	Лист	№ докум.	Подп.	Дата		

3.5 Реализация аукционов

Создание и управление аукционами реализовано в сервисе AuctionServiceImpl. Создать аукцион может только пользователь с ролью OWNER. Если пользователь с другой ролью пытается создать аукцион, система отклоняет действие.

При создании аукциона указываются название, описание, стартовая цена, дата начала и дата окончания. Система проверяет корректность временного периода. Дата начала не должна быть позже даты окончания.

После проверки данных определяется начальный статус аукциона. Если дата начала находится в будущем, аукцион получает статус DRAFT. Если дата начала уже наступила, аукцион получает статус ACTIVE.

Для обновления статусов используется метод refreshStatuses. Он проверяет аукционы и изменяет их состояние в зависимости от текущего времени. Если время окончания уже наступило, аукцион получает статус CLOSED. Если время начала наступило, а аукцион находится в состоянии DRAFT, он переводится в состояние ACTIVE.

Владелец может закрыть только свой аукцион. Администратор может закрыть любой аукцион. После закрытия аукцион получает статус CLOSED. Форма создания аукциона показана на рисунке 9.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						28
Изм.	Лист	№ докум.	Подп.	Дата		

Рисунок 9 – Страница создания аукциона

3.6 Реализация системы ставок

Размещение ставок реализовано в сервисе `BidServiceImpl`. Пользователь указывает идентификатор аукциона и сумму ставки. После этого система получает аукцион, пользователя и текущую цену.

Перед сохранением ставки выполняется несколько проверок. Сначала проверяется, что учётная запись пользователя активна. Если пользователь заблокирован, размещение ставки запрещается.

Далее вызывается стратегия проверки ставки `BidValidationStrategy`.

Реализация `DefaultBidValidationStrategy` проверяет следующие условия:

- аукцион должен быть активным;
- время окончания аукциона не должно истечь;
- владелец не может делать ставки на свой аукцион;

– новая ставка должна быть больше текущей цены.

Если все проверки пройдены, создаётся объект Bid. В нём сохраняются сумма ставки, время размещения, пользователь и аукцион. После этого ставка сохраняется в базе данных.

Текущая цена аукциона определяется по максимальной ставке. Если ставок ещё нет, текущей ценой считается стартовая цена аукциона. Интерфейс размещения ставки представлен на рисунке 10.

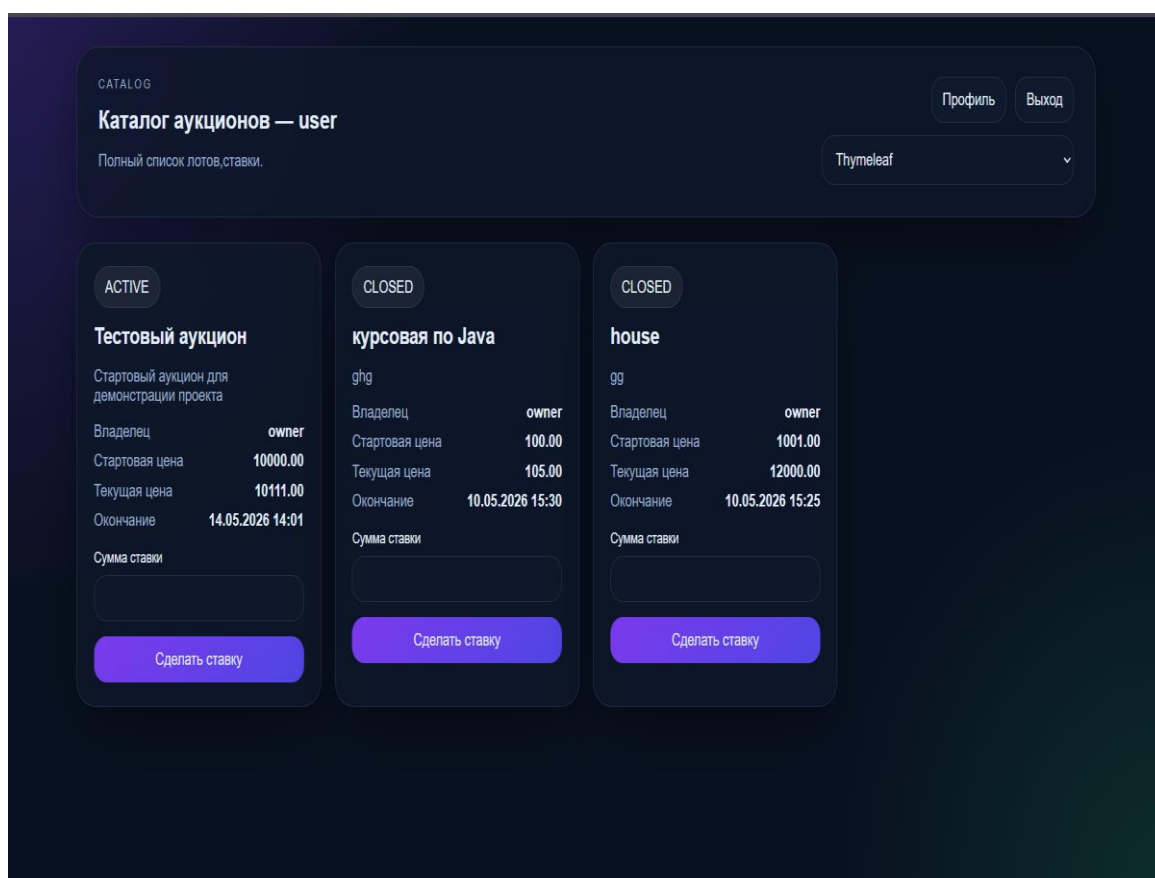


Рисунок 10 – Страница размещения ставки

3.7 Реализация личного кабинета

Для каждого пользователя предусмотрен личный кабинет, в котором он может просматривать информацию о своих ставках и участии в аукционах. Также в личном кабинете отображается история участия в торгах.

Владельцы аукционов имеют доступ к панели управления своими лотами, где они могут просматривать информацию о своих аукционах и управлять их статусами. Личный кабинет пользователя показан на рисунке 11.

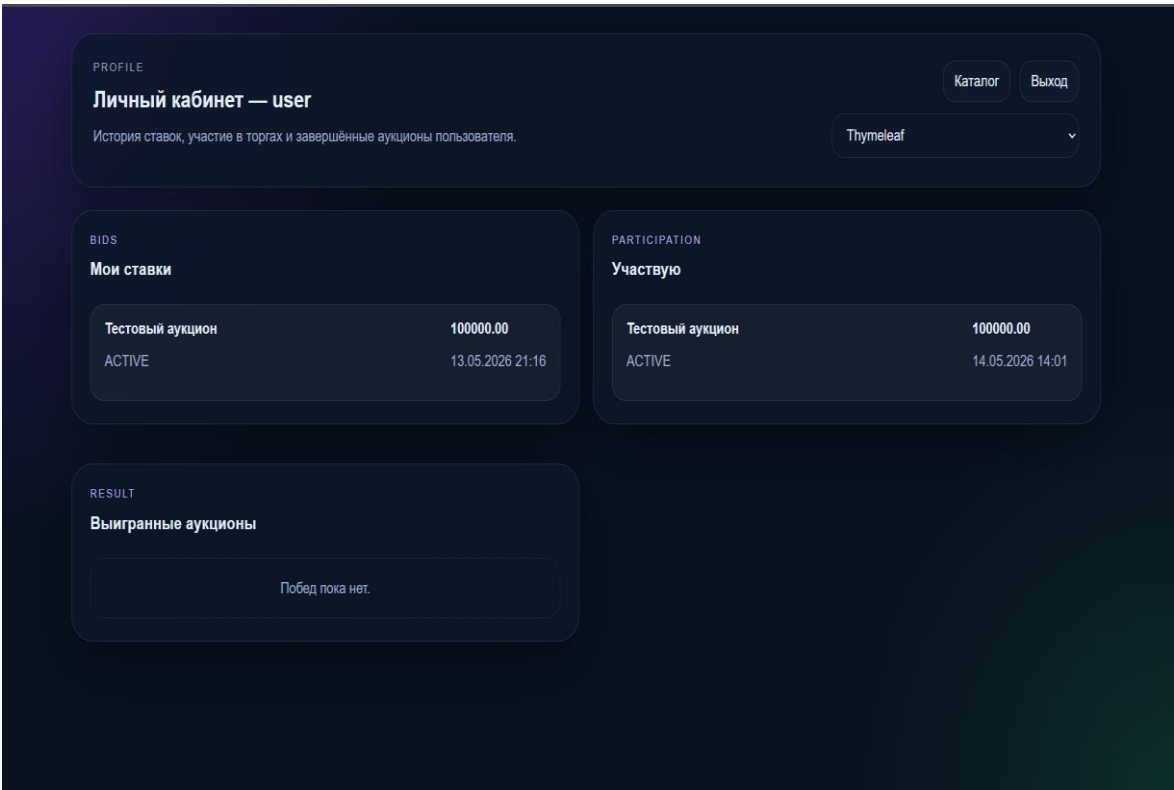


Рисунок 11 – Личный кабинет пользователя

3.8 Панель владельца

Панель владельца аукционов включает в себя функции для создания новых аукционов, изменения их статусов и просмотра ставок. Владелец может закрыть аукцион, тем самым завершив торги, а также изменить параметры аукциона, такие как стартовая цена и описание.

Владельцы могут просматривать информацию о сделанных ставках и видеть текущую цену на их аукционы. Панель управления владельца аукционов показана на рисунке 12.

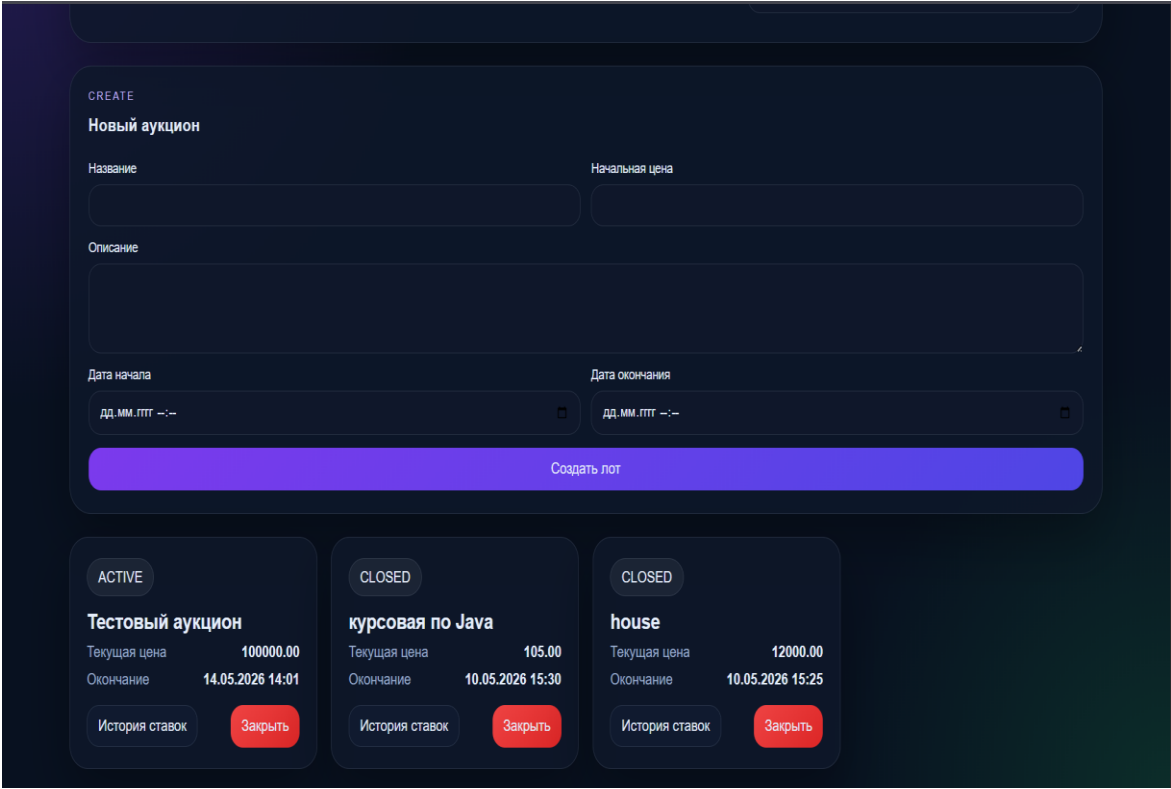


Рисунок 12 – Панель владельца

3.9 Административная панель

Административная панель предоставляет доступ к функциям управления системой. Администратор может просматривать список пользователей и аукционов, удалять пользователей, а также управлять тестами, проверяя корректность их работы. Административная панель системы представлена на рисунке 13.

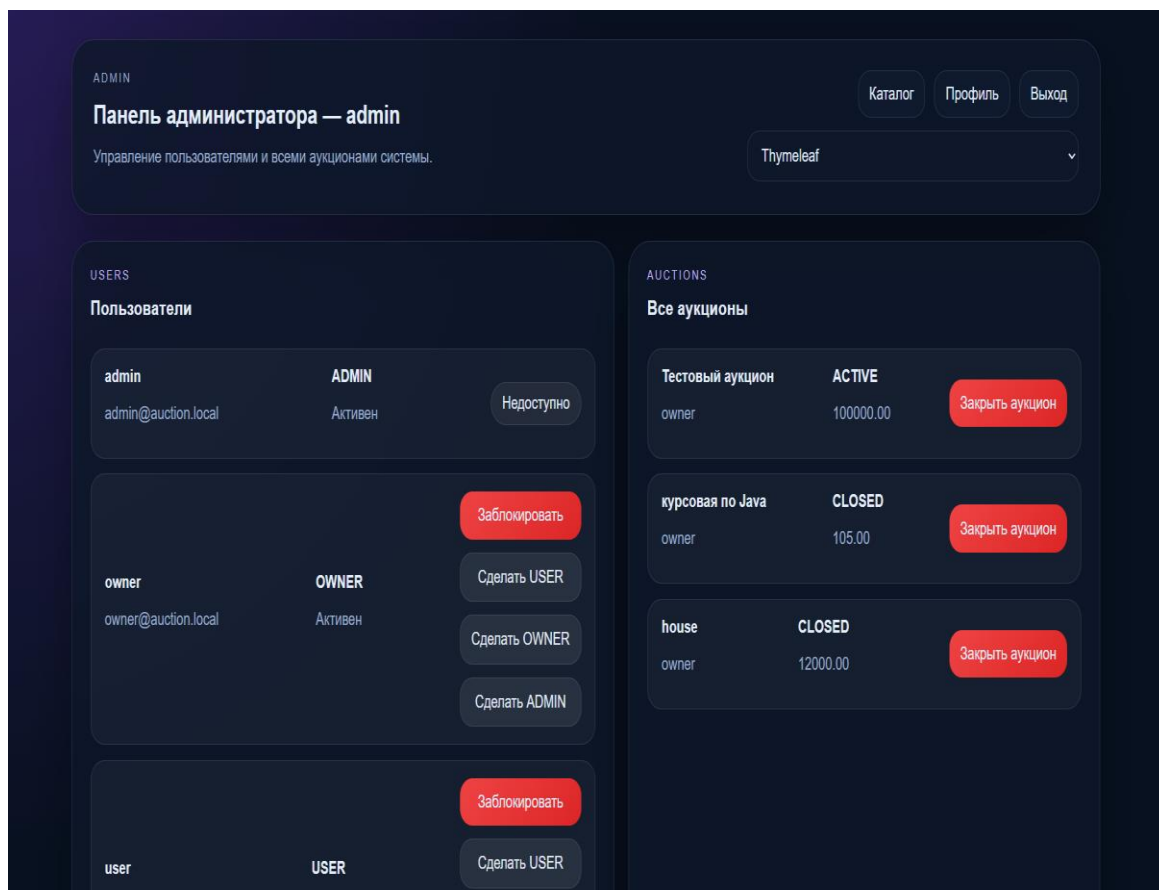


Рисунок 13 – Панель администратора

3.10 Реализация трёх web-интерфейсов

Одним из требований технического задания является реализация трёх разных движков web-интерфейса^[2]. В проекте используются Thymeleaf, Mustache и FreeMarker.

Для каждого движка созданы отдельные шаблоны страниц. Шаблоны расположены в каталогах:

- templates/thymeleaf;
- templates/mustache;
- templates/freemarker.

Все три варианта интерфейса используют одну и ту же программную логику. Отличается только способ формирования HTML-страницы. Это позволяет показать, что логика приложения отделена от уровня отображения.

В приложении реализованы страницы входа, регистрации, списка аукционов, личного кабинета, панели владельца и административной панели. Пользователь может работать с разными вариантами интерфейса.

Основные адреса страниц имеют следующий вид:

- thymeleaf/auctions;
- mustache/auctions;
- freemarker/auctions.

Реализация трёх web-интерфейсов представлена на рисунке 14.

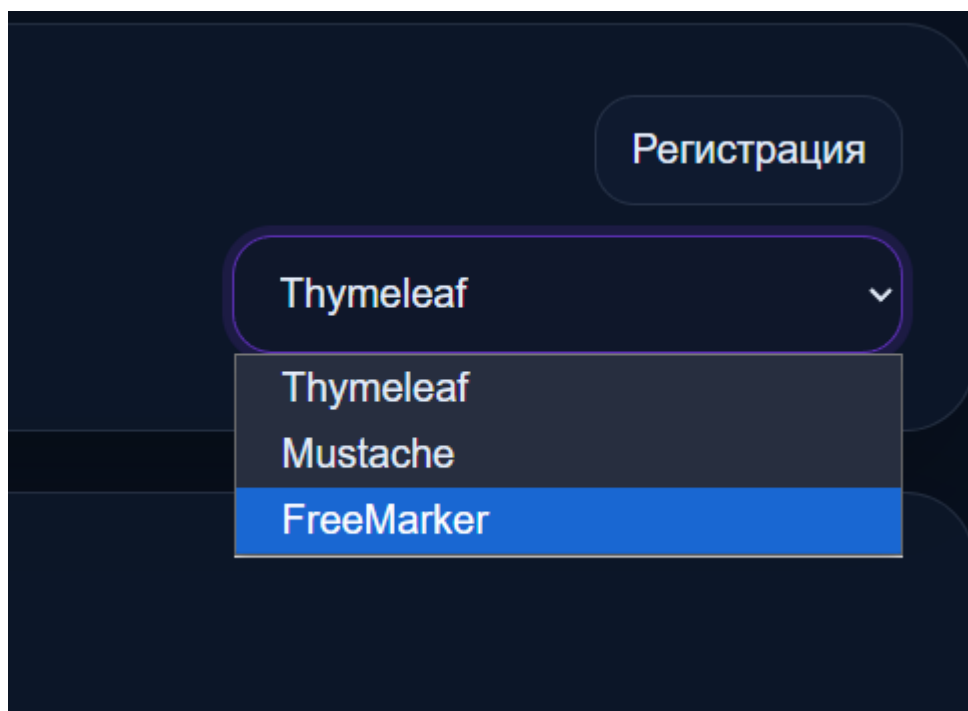


Рисунок 14 – Пример выбора веб-движка

4 Тестирование

В процессе разработки веб-приложения «Аукцион» было проведено тестирование основных компонентов системы. Целью тестирования являлась проверка корректности регистрации пользователей, создания аукционов, размещения ставок, работы ролевой модели, использования паттернов проектирования и поддержки нескольких движков web-интерфейса.

Для проверки программной логики использовались unit-тесты, написанные с применением JUnit 5 и Mockito. Тестирование выполнялось по отдельным компонентам, что позволило проверить работу сервисов, стратегий и фабрик без запуска всего приложения.

4.1 Тестирование модуля пользователей и регистрации

Первым этапом было проведено тестирование модуля пользователей, отвечающего за регистрацию новых учетных записей, проверку уникальности логина и электронной почты, назначение роли и шифрование пароля.

Тестирование выполнялось в классе UserServiceImplTest. В методе registerShouldCreateUserWithUserRole() проверялась успешная регистрация нового пользователя. Для этого создавался объект RegisterRequest с логином, электронной почтой и паролем. С помощью Mockito имитировалась работа UserRepository и PasswordEncoder. После вызова метода register() проверялось, что пользователь получает роль USER, включенное состояние учетной записи и зашифрованный пароль.

Также проверялась обработка ошибочных ситуаций. Метод registerShouldThrowWhenUsernameExists() тестировал попытку регистрации пользователя с уже существующим логином. Метод registerShouldThrowWhenEmailExists() проверял ситуацию, когда указанная электронная почта уже используется другим пользователем. В обоих случаях система должна выбрасывать исключение и не сохранять некорректные данные.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						35
Изм.	Лист	№ докум.	Подп.	Дата		

Результаты тестирования подтвердили, что модуль регистрации корректно создает новых пользователей, проверяет уникальность данных и обеспечивает безопасное хранение паролей. Результаты тестирования модуля пользователей и регистрации представлены на рисунках 15–16.

```

32
33     @Test & VLEKS
34     void registerShouldCreateUserWithUserRole() {
35         RegisterRequest request = new RegisterRequest();
36         request.setUsername("newuser");
37         request.setEmail("newuser@test.local");
38         request.setPassword("123456");
39
40         when(userRepository.existsByUsername("newuser")).thenReturn( t: false);
41         when(userRepository.existsByEmail("newuser@test.local")).thenReturn( t: false);
42         when(passwordEncoder.encode( rawPassword: "123456")).thenReturn( t: "encoded-password");
43         when(userRepository.save(any(User.class))).thenReturn( invocationOnMock invocation -> invocation.getArgument( t: 0));
44
45         User savedUser = userService.register(request);
46
47         assertEquals( expected: "newuser", savedUser.getUsername());
48         assertEquals( expected: "newuser@test.local", savedUser.getEmail());
49         assertEquals( expected: "encoded-password", savedUser.getPassword());
50         assertEquals(Role.USER, savedUser.getRole());
51         assertTrue(savedUser.isEnabled());
52
53         ArgumentCaptor<User> captor = ArgumentCaptor.forClass(User.class);
54         verify(userRepository).save(captor.capture());
55         assertEquals(Role.USER, captor.getValue().getRole());
56     }
57
58     @Test & VLEKS

```

Рисунок 15 – Тестирование регистрации пользователя

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						36
Изм.	Лист	№ докум.	Подп.	Дата		

```

58  @Test & VLEKS
59  void registerShouldThrowWhenUsernameExists() {
60      RegisterRequest request = new RegisterRequest();
61      request.setUsername("existing");
62      request.setEmail("mail@test.local");
63      request.setPassword("123456");
64
65      when(userRepository.existsByUsername("existing")).thenReturn(true);
66
67      IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> userService.register(request))
68      assertEquals( expected: "Пользователь с таким логином уже существует", exception.getMessage());
69  }
70
71  @Test & VLEKS
72  void registerShouldThrowWhenEmailExists() {
73      RegisterRequest request = new RegisterRequest();
74      request.setUsername("user1");
75      request.setEmail("existing@test.local");
76      request.setPassword("123456");
77
78      when(userRepository.existsByUsername("user1")).thenReturn(false);
79      when(userRepository.existsByEmail("existing@test.local")).thenReturn(true);
80
81      IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> userService.register(request))
82      assertEquals( expected: "Пользователь с таким email уже существует", exception.getMessage());
83  }
84

```

Рисунок 16 – Тестирование проверки уникальности логина и электронной почты

4.2 Тестирование модуля аукционов

Следующим этапом было протестировано создание аукционов. Данный компонент отвечает за добавление новых лотов, установку стартовой цены, указание периода проведения торгов, назначение владельца и определение статуса аукциона.

Тестирование проводилось в классе AuctionServiceImplTest. В методе createShouldAllowOwner() проверялось, что пользователь с ролью OWNER может создать новый аукцион. Для теста создавался объект AuctionRequest с названием, описанием, стартовой ценой, датой начала и датой окончания торгов. После вызова метода create() проверялось, что созданный аукцион получил корректное название, владельца и статус ACTIVE.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						37
Изм.	Лист	№ докум.	Подп.	Дата		

Отдельно проверялось ограничение доступа. Метод `createShouldRejectAdmin()` моделировал ситуацию, при которой пользователь с ролью ADMIN пытается создать аукцион. Система должна отклонить это действие, так как создание лотов разрешено только владельцам.

Также была протестирована проверка дат проведения аукциона. В методе `createShouldRejectWhenStartTimeAfterEndTime()` задавалась дата начала позже даты окончания. В результате система должна выбросить исключение и не сохранить аукцион с некорректным временным интервалом.

Результаты тестирования подтвердили, что сервис аукционов корректно создает лоты, проверяет роль пользователя и не допускает сохранения ошибочных данных. Тестирование модуля аукционов показано на рисунках 17–18.

```

42 void createShouldAllowOwner() {
43     AuctionRequest request = new AuctionRequest();
44     request.setTitle("Ноутбук");
45     request.setDescription("Игровой ноутбук");
46     request.setStartPrice(new BigDecimal("10000.00"));
47     request.setStartTime(LocalDateTime.now().minusMinutes(5));
48     request.setEndTime(LocalDateTime.now().plusDays(1));
49
50     User owner = User.builder()
51         .id(1L)
52         .username("owner")
53         .role(Role.OWNER)
54         .enabled(true)
55         .build();
56
57     when(userService.getByUsername("owner")).thenReturn(owner);
58     when(auctionRepository.save(any(Auction.class))).thenReturn(invocationOnMock invocation -> invocation.
59         getArgument(0));
60
61     Auction auction = auctionService.create(request, username: "owner");
62
63     assertEquals("Ноутбук", auction.getTitle());
64     assertEquals(owner, auction.getOwner());
65     assertEquals(AuctionStatus.ACTIVE, auction.getStatus());
66
67     ArgumentCaptor<Auction> captor = ArgumentCaptor.forClass(Auction.class);
68     verify(auctionRepository).save(captor.capture());
69     assertEquals(Role.OWNER, captor.getValue().getOwner().getRole());
70 }

```

Рисунок 17 – Тестирование создания аукциона владельцем

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		38

```

72 void createShouldRejectAdmin() {
73     AuctionRequest request = new AuctionRequest();
74     request.setTitle("Товар");
75     request.setDescription("Описание");
76     request.setStartPrice(new BigDecimal( val: "1000.00"));
77     request.setStartTime(LocalDateTime.now());
78     request.setEndTime(LocalDateTime.now().plusDays(1));
79
80     User admin = User.builder()
81         .id(2L)
82         .username("admin")
83         .role(Role.ADMIN)
84         .enabled(true)
85         .build();
86
87     when(userService.getByUsername("admin")).thenReturn(admin);
88
89     IllegalStateException exception = assertThrows(IllegalStateException.class, () -> auctionService.create(request,
90     assertEquals( expected: "Создавать аукционы может только владелец", exception.getMessage());
91 }
92
93 @Test & VLEKS
94 void createShouldRejectWhenStartTimeAfterEndTime() {
95     AuctionRequest request = new AuctionRequest();
96     request.setTitle("Товар");
97     request.setDescription("Описание");
98     request.setStartPrice(new BigDecimal( val: "1000.00"));
99     request.setStartTime(LocalDateTime.now().plusDays(2));
100    request.setEndTime(LocalDateTime.now().plusDays(1));
101
102    User owner = User.builder()
103        .id(1L)
104        .username("owner")
105        .role(Role.OWNER)

```

Рисунок 18 – Тестирование ограничений при создании аукциона

4.3 Тестирование модуля ставок

Следующим этапом было проведено тестирование модуля ставок. Данный компонент отвечает за размещение ставок пользователями, проверку активности учетной записи, определение текущей цены аукциона и сохранение ставки в базе данных.

Тестирование выполнялось в классе BidServiceImplTest. В методе placeBidShouldSaveBid() проверялось успешное размещение ставки пользователем. Для теста создавались объект пользователя, объект аукциона и объект BidRequest, содержащий идентификатор аукциона и сумму ставки. С

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						39
Изм.	Лист	№ докум.	Подп.	Дата		

помощью Mockito имитировалась работа репозитория и сервисов. После вызова метода `placeBid()` проверялось, что ставка была создана с правильной суммой, пользователем и аукционом.

Также была проверена ситуация, когда пользователь заблокирован. В методе `placeBidShouldRejectDisabledUser()` создавался пользователь с отключенной учетной записью. При попытке разместить ставку система должна выбросить исключение и не сохранить данные.

Результаты тестирования подтвердили, что модуль ставок корректно сохраняет допустимые ставки и запрещает размещение ставок от заблокированных пользователей. Тестирование размещения ставки представлено на рисунке 19. Тестирование запрета ставки для заблокированного пользователя показано на рисунке 20.

```

46  @Test  @VLEKS
47  void placeBidShouldSaveBidForEnabledUser() {
48      User owner = User.builder()
49          .id(10L)
50          .username("owner")
51          .role(Role.OWNER)
52          .enabled(true)
53          .build();
54
55      Auction auction = Auction.builder()
56          .id(1L)
57          .title("Лот")
58          .startPrice(new BigDecimal( val: "100.00"))
59          .status(AuctionStatus.ACTIVE)
60          .endTime(LocalDate.now().plusDays(1))
61          .owner(owner)
62          .build();
63
64      User bidder = User.builder()
65          .id(20L)
66          .username("user")
67          .role(Role.USER)
68          .enabled(true)
69          .build();
70
71      BidRequest request = new BidRequest();
72      request.setAuctionId(1L);
73      request.setAmount(new BigDecimal( val: "150.00"));
74
75      when(auctionService.getById(1L)).thenReturn(auction);
76      when(userService.getByUsername("user")).thenReturn(bidder);
77      when(bidRepository.findTopByAuctionOrderByAmountDesc(auction)).thenReturn( Optional.empty());

```

Рисунок 19 – Тестирование размещения ставки

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						40
Изм.	Лист	№ докум.	Подп.	Дата		


```

87
88     @Test & VLEKS
89     void placeBidShouldRejectDisabledUser() {
90         User owner = User.builder()
91             .id(10L)
92             .username("owner")
93             .role(Role.OWNER)
94             .enabled(true)
95             .build();
96
97         Auction auction = Auction.builder()
98             .id(1L)
99             .title("Лот")
100             .startPrice(new BigDecimal( val: "100.00"))
101             .status(AuctionStatus.ACTIVE)
102             .endTime(LocalDate.now().plusDays(1))
103             .owner(owner)
104             .build();
105
106         User blockedUser = User.builder()
107             .id(20L)
108             .username("blocked")
109             .role(Role.USER)
110             .enabled(false)
111             .build();
112
113         BidRequest request = new BidRequest();
114         request.setAuctionId(1L);
115         request.setAmount(new BigDecimal( val: "150.00"));
116
117         when(auctionService.getById(1L)).thenReturn(auction);
118         when(userService.getUsername("blocked")).thenReturn(blockedUser);
119

```

Рисунок 20 – Тестирование запрета ставки для заблокированного пользователя

4.4 Тестирование паттернов проектирования и интерфейсов

Отдельно было проведено тестирование проверки правил размещения ставок. Данный механизм реализован с применением паттерна Strategy. В проекте используется интерфейс BidValidationStrategy и его реализация DefaultBidValidationStrategy.

Тестирование выполнялось в классе DefaultBidValidationStrategyTest. Проверялись основные ограничения, которые должны выполняться перед сохранением ставки. В частности, система должна запрещать размещение ставки на закрытый аукцион, запрещать владельцу делать ставку на

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		41

собственный лот, а также отклонять ставку, если ее сумма меньше или равна текущей цене.

Также проверялся успешный сценарий, при котором аукцион находится в статусе ACTIVE, пользователь не является владельцем лота, а сумма ставки превышает текущую цену. В этом случае проверка должна завершиться без ошибки.

Результаты тестирования подтвердили, что стратегия проверки ставок работает корректно и не допускает нарушения правил проведения аукциона. Тестирование стратегии проверки ставок представлено на рисунке 21.

```
8
9
10 @Test & VLEKS *
11 void validateShouldAllowCorrectBid() {
12     Auction auction = buildAuction(AuctionStatus.ACTIVE, ownerUsername: "owner", LocalDateTime.now().plusDays(1));
13
14     assertDoesNotThrow(() ->
15         strategy.validate(auction, new BigDecimal( val: "100.00"), new BigDecimal( val: "150.00"),
16             bidderUsername: "user"));
17 }
18
19 private Auction buildAuction(AuctionStatus status, String ownerUsername, LocalDateTime endTime) { 4 usages & VLEKS
20     User owner = User.builder()
21         .id(1L)
22         .username(ownerUsername)
23         .role(Role.OWNER)
24         .enabled(true)
25         .build();
26
27     return Auction.builder()
28         .id(1L)
29         .title("Товар")
30         .description("Описание")
31         .startPrice(new BigDecimal( val: "100.00"))
32         .startTime(LocalDateTime.now().minusDays(1))
33         .endTime(endTime)
34         .status(status)
35         .owner(owner)
36         .build();
37 }
38 }
```

Рисунок 21 – Тестирование стратегии проверки ставок

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						42
Изм.	Лист	№ докум.	Подп.	Дата		

4.5 Тестирование фабрики карточек аукционов

Для проверки паттерна Factory было выполнено тестирование класса AuctionCardFactory. Данный класс отвечает за создание объекта AuctionCardDto, который используется для отображения информации об аукционе на страницах приложения.

Тестирование выполнялось в классе AuctionCardFactoryTest. В ходе теста создавался объект аукциона с заполненными полями: названием, описанием, стартовой ценой, владельцем, статусом, датой начала и датой окончания. После передачи аукциона в фабрику проверялось, что созданный DTO-объект содержит корректные данные.

Результаты тестирования подтвердили, что фабрика правильно формирует карточку аукциона и может использоваться для передачи данных в web-интерфейс. Тестирование фабрики карточек аукционов представлено на рисунке 21.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						43
Изм.	Лист	№ докум.	Подп.	Дата		

```

@Test
void createShouldBuildAuctionCardDto() {
    User owner = User.builder()
        .id(1L)
        .username("owner")
        .role(Role.OWNER)
        .enabled(true)
        .build();

    Auction auction = Auction.builder()
        .id(5L)
        .title("Смартфон")
        .description("Новый смартфон")
        .startPrice(new BigDecimal("10000.00"))
        .startTime(LocalDateTime.of(year: 2026, month: 1, dayOfMonth: 1, hour: 10, minute: 0))
        .endTime(LocalDateTime.of(year: 2026, month: 1, dayOfMonth: 10, hour: 10, minute: 0))
        .status(AuctionStatus.ACTIVE)
        .owner(owner)
        .build();

    AuctionCardFactory factory = new AuctionCardFactory();
    AuctionCardDto dto = factory.create(auction, new BigDecimal("12000.00"));

    assertEquals(expected: 5L, dto.getId());
    assertEquals(expected: "Смартфон", dto.getTitle());
    assertEquals(expected: "Новый смартфон", dto.getDescription());
    assertEquals(new BigDecimal("10000.00"), dto.getStartPrice());
    assertEquals(new BigDecimal("12000.00"), dto.getCurrentPrice());
    assertEquals(expected: "owner", dto.getOwnerUsername());
    assertEquals(expected: "ACTIVE", dto.getStatus());
    assertEquals(LocalDateTime.of(year: 2026, month: 1, dayOfMonth: 1, hour: 10, minute: 0), dto.getStartTime());
    assertEquals(LocalDateTime.of(year: 2026, month: 1, dayOfMonth: 10, hour: 10, minute: 0), dto.getEndTime());
}

```

Рисунок 22 – Тестирование фабрики карточек аукционов

4.6 Результаты тестирования

По результатам unit-тестирования было установлено, что основные функции веб-приложения работают корректно. Регистрация пользователей, создание аукционов, размещение ставок, проверка ролей и работа паттернов проектирования выполняются в соответствии с техническим заданием. Результаты unit-тестов представлены в таблице 1 и на рисунке 23.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						44
Изм.	Лист	№ докум.	Подп.	Дата		

Таблица 1 – Результаты unit-тестирования

№	Компонент	Проверка	Результат
1	Модуль пользователей	Регистрация нового пользователя	Успешно
2	Модуль пользователей	Проверка уникальности логина и email	Успешно
3	Модуль аукционов	Создание аукциона владельцем	Успешно
4	Модуль аукционов	Ограничение роли	Успешно
5	Модуль ставок	Размещение ставки	Успешно
6	Модуль ставок	Заблокированный пользователь	Успешно
7	Паттерн Factory	Создание DTO карточки аукциона	Успешно
8	Паттерн Strategy	Проверка корректности ставок	Успешно

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.auction.pattern.factory.AuctionCardFactoryTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.096 s -- in com.auction.pattern.factory.AuctionCardFactoryTest
[INFO] Running com.auction.pattern.strategy.DefaultBidValidationStrategyTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in com.auction.pattern.strategy.DefaultBidValidationStrategyTest
[INFO] Running com.auction.service.impl.AuctionServiceImplTest
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.793 s -- in com.auction.service.impl.AuctionServiceImplTest
[INFO] Running com.auction.service.impl.BidServiceImplTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.112 s -- in com.auction.service.impl.BidServiceImplTest
[INFO] Running com.auction.service.impl.UserServiceImplTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.146 s -- in com.auction.service.impl.UserServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.180 s
[INFO] Finished at: 2026-05-22T12:27:47+03:00
[INFO] -----
C:\auction>

```

Рисунок 23 – Результаты запуска unit-тестов

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						46
Изм.	Лист	№ докум.	Подп.	Дата		

Заключение

В ходе выполнения курсовой работы был разработан веб-сервис для проведения онлайн-аукционов, который включает в себя функциональность регистрации и авторизации пользователей, создание и управление аукционами, размещение ставок и мониторинг истории торгов. Приложение включает в себя три ключевых роли: обычный пользователь, владелец аукционов и администратор, каждая из которых имеет свои права и возможности в системе.

Использование фреймворка Spring Boot позволило создать гибкую и расширяемую архитектуру приложения, обеспечив взаимодействие с базой данных через Spring Data JPA и безопасность с помощью Spring Security. Приложение обеспечивает надежную работу с базой данных, правильное хранение данных о пользователях, аукционах и ставках, а также динамическое обновление информации о текущих торгах.

Реализация интерфейса с использованием шаблонизаторов позволила легко адаптировать внешний вид приложения и продемонстрировать гибкость архитектуры.

Проведенное тестирование подтвердило корректную работу всех основных функций, таких как регистрация пользователей, создание аукционов, размещение ставок, а также безопасность приложения. Тестирование пользовательского интерфейса продемонстрировало интуитивно понятное взаимодействие с системой, а тесты на безопасность подтвердили защиту данных и эффективную работу механизмов аутентификации и авторизации.

Таким образом, разработанное веб-приложение "Аукцион" полностью соответствует поставленным целям и задачам, предоставляя пользователю функциональный и безопасный интерфейс для участия в онлайн-аукционах.

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
						47
Изм.	Лист	№ докум.	Подп.	Дата		

Список используемой литературы

1. Документация по Java Электронный ресурс Электронный ресурс: справочное руководство / Oracle Corporation. — Электрон. текстовые данные. — Режим доступа: <https://docs.oracle.com/javase/>
2. Документация по Java Электронный ресурс Электронный ресурс: справочное руководство / Oracle Corporation. — Электрон. текстовые данные. — Режим доступа: <https://docs.oracle.com/javase/>
3. Документация по RedExpert [Электронный ресурс]: справочное руководство / RedDataBase. — Электрон. текстовые данные. — Режим доступа: <https://www.red-database.com/redexpert>
4. Документация по базе данных RedDataBase [Электронный ресурс]: справочное руководство / RedDataBase. — Электрон. текстовые данные. — Режим доступа: <https://www.red-database.com/documentation>

					МИВУ 09.03.02 – 00.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		48